

For this assignment, we were provided with skeleton code for a graph percolation algorithm. This assignment investigates the use of OpenMP for parallelising the percolation algorithm.

Parallelism over vertices

Prior to adding the `-fopenmp` flag, the execution time was 0.0830 with parameters `Np = Ngraphs = 1` and `Nvert = 5000`. Adding this flag more than halved the execution time, bringing it down to 0.0341 seconds.

The next step was to identify a possible data race in the code. The nested for-loop for edge generation between vertices had a parallelised outer for loop, with the inner loop using vertices that may have overlaps with values of `i` across other threads. This highlights the potential issues:

- Threads can increment the same `Ncon[j]` array element concurrently leading to lost updates.
- Threads can write to the same `Lcon` segments leading to incorrect adjacency data.

The solution to these issues is the use of `#pragma omp critical` around the update block within the conditional `if (xi < Pcon)`. This protects shared modifications, updating `Ncon`, `Lcon` one thread at a time, serialising the shared updates. Whilst the runtime stayed the same at 0.033 seconds, the output showed reproducibility on a defined schedule and number of threads.

The investigation then evaluates how OpenMP scheduling can affect loop performance. A base time was set by using chunk size `Nvert / 4` ($= 1250$), with time 0.0055 seconds. The expectation was that as chunk size decreases, so would the timings. The results were as follows:

Schedule Type	Time (s)
<code>schedule(static, Nvert/4)</code>	0.055
<code>schedule(static, 1000)</code>	0.049
<code>schedule(static, 100)</code>	0.032
<code>schedule(static, 10)</code>	0.032
<code>schedule(dynamic, 100)</code>	0.034
<code>schedule(guided, 100)</code>	0.035

Table 1: Execution times for different OpenMP scheduling strategies

Static scheduling had the best performance, with dynamic and guided scheduling being 0.02 seconds and 0.03 seconds respectively from the static counterpart. This is likely a result of incurred computational cost, such as synchronisation.

Program variables were then changed, such that `Np = 8`, `Ngraphs = 20` to calculate statistics across several values of P . Environment variables in file `perc.slurm` were changed such that the number of CPUs and threads were 4. The program was executed using the SLURM batch processing system (via SCRTP), with an execution time of 25.8741 seconds.

Parallelism over P

The `omp parallel` directive and inner loop pragmas were then removed from the vertex loop. This task aimed to parallelise over P , the loop that handles vertex connectivity probabilities. The pragma directive `#pragma omp parallel for default(shared) private(igraph,i,j,xi,Pcon)` was added, keeping `igraph`, `i`, `j`, `xi`, `Pcon` as private copies per thread. The parallelism was tested by changing the CPU and thread count, whilst keeping them equal, from values 1 to 4 (i.e. 1 CPU, 1 thread).

Threads	Time (s)	Speedup
1	33.9648	1.000×
2	21.9991	1.545×
3	12.6270	2.690×
4	11.0540	3.073×

Table 2: Parallelism over P , unordered output

By comparing the parallelised vertex time with the 4 CPU, 4 threads time of the parallelised P -loop, the parallelised P -loop was ≈ 2.341 times faster. In this approach, the output is not in a convenient order. To combat this, the `ordered` keyword is added to the end of the pragma. In addition, `#pragma omp ordered` is added to ensure the threads print each statement in serial order.

Threads	Time (s)	Speedup
1	34.0668	1.000×
2	28.2715	1.205×
3	24.1951	1.408×
4	21.1035	1.614×

Table 3: Parallelism over P , ordered output using `ordered` keyword

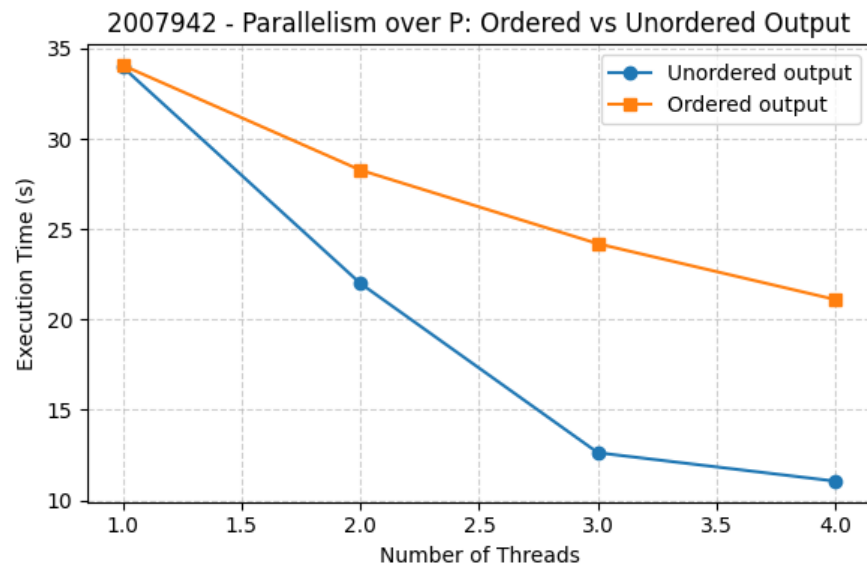


Figure 1: Comparison of execution times for ordered vs unordered output in parallelism over P .

Figure 1 shows that both whilst the unordered and ordered outputs decrease in execution time, the ordered output has a shallower slope. The use of `#pragma omp ordered` results in a synchronisation overhead as each thread has to wait to print, reducing the effectiveness of the parallelisation.

Parallelism over graphs

This task parallelises over graphs rather than over the P -loop. All of the OpenMP pragmas and directives related to the P -loop task were deleted. To parallelise the graph generation, the following pragma was added: `#pragma omp parallel for default(shared) private(Ncon, Lcon, i, j, xi, lclus, nclus, Pcon) reduction(+:avlclus, avnclus)`, with each thread generating their own graphs, arrays and cluster statistics. The use of reduction evaluates results in separate local copies then sums the results together at the end of the parallel evaluation. The batch processing is used again to evaluate the performance across several CPUs and threads.

Threads	Time (s)	Speedup
1	14.3884	1.00×
2	7.1848	2.00×
3	5.0346	2.86×
4	3.6003	4.00×

Table 4: Parallelism over graphs

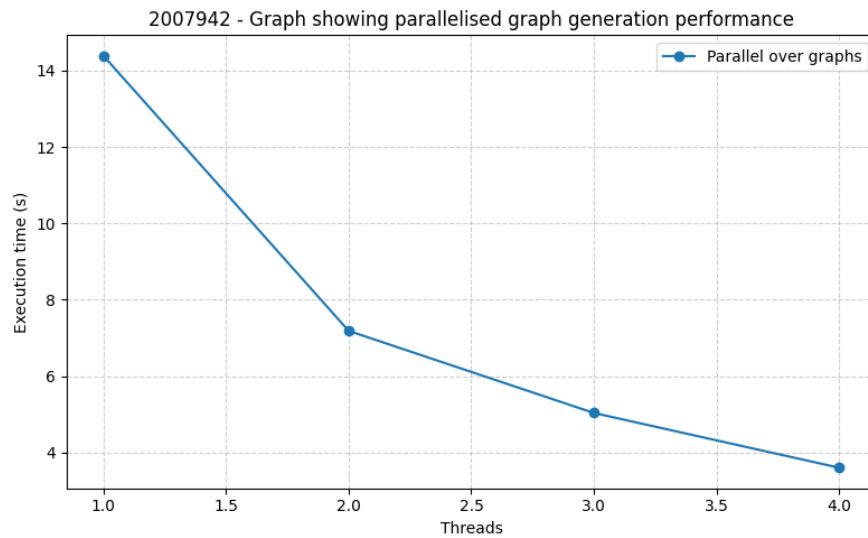


Figure 2: Plot showing how execution time changes with increasing number of threads for evaluating parallelism over graph generation.

Parallelism over graphs yields significant performance improvements compared to parallelising the P -loop, with an ≈ 3 times greater efficiency at 4 threads than the unordered P -loop. This is a result of several factors:

- Parallel overheads: for the parallelised P -loop, threads are perked once per `ip` loop whereas the parallelised graph loop forks 8 times.
- Distribution of work to threads: parallel P has 8 coarse tasks, leading to idle threading, whereas the graph parallel has even workload distribution and thus greater efficiency.
- Synchronisation: in the parallel P loop, the use of the `ordered` keyword serialises the print statements; for the parallelised graph code, the use of `reduction` sums values at the end, minimising synchronisation cost.

Nested Parallelism

This final section combines both approaches, parallelising over both P and graph generation. We redefine $N_p = 2$ and change environment variables in `perc.slurm` such that `OMP_NUM_THREADS = 2`, `OMP_NESTED = TRUE` with the use of 4 CPUs. This implementation used the following:

- Two levels of parallelism are used, one on the outer P -loop and one across the graph loop. Each loop using `num_threads(2)` executing 4 threads in parallel. As a result, each `Pcon` value generates graphs in parallel.
- All graph and thread related variables were declared as private in the pragma, preventing lost update errors. Reductions are used once again for local summation followed by a final result calculation.
- Since graphs are generated locally per thread, eliminating issues with shared memory bandwidth.

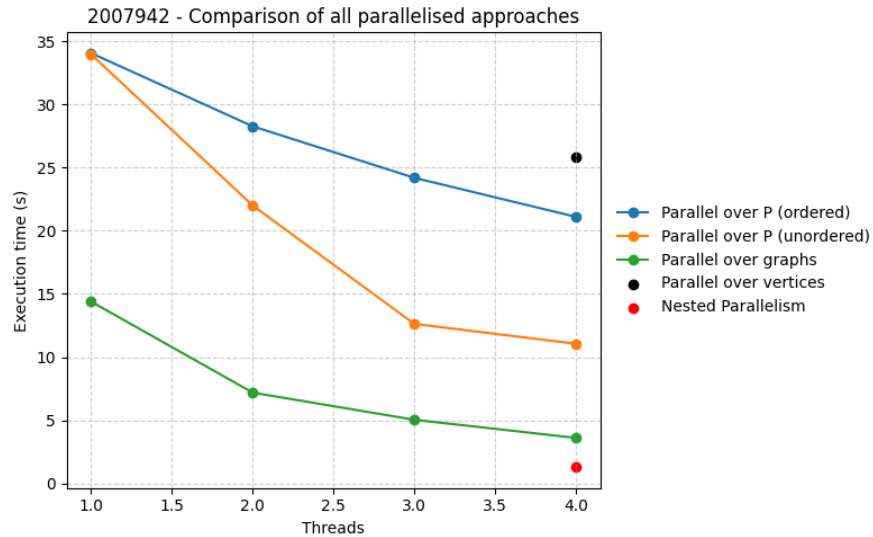


Figure 3: Comparison of execution times for the collected data across each experiment for this assignment.

The nested parallelism approach (shown in red) had the optimal performance between all approaches.